

Testing & Assessment Document

Supervised Process Intelligence Praktikum SS 2026

Dr. Alessandro Berti

Lehrstuhl für Process and Data Science
RWTH Aachen University

By the end of this session, you should be able to:

- Explain what the **Assessment Document** should contain for Milestone 6.
- Design a convincing evaluation for a supervised process intelligence project.
- Select metrics, baselines, plots, and sanity checks for your specific assignment.
- Document testing evidence and reproducibility in a way that another person can verify.
- Write a critical assessment: strong points, limitations, threats to validity, and next steps.

- **Milestone 6: Testing and Assessment** is the final milestone of the project work.
- Expected outputs:
 - **Assessment document.**
 - **Phase review document.**
 - **Code and tests** in the Git repository.
- The assessment document should connect three things:
 - the assignment goal,
 - the implemented software artifact,
 - the empirical evidence that the solution works, where it works, and where it does not.

Purpose of the Assessment Document

- The document is **not only a README** and not only a collection of screenshots.
- It should answer the central question:
 - **How well does your implemented approach solve the assigned process intelligence problem?**
- It should provide evidence about:
 - quality of predictions or explanations,
 - behavior under different prefixes, parameters, groups, or data conditions,
 - testing strategy and correctness checks,
 - reproducibility and usability of the prototype,
 - limitations and risks.

Assessment vs. Implementation Documentation

Repository / README	Technical documentation	Assessment document
How to install and run the software.	How the modules, data flow, and design decisions are organized.	What the experiments show and how to interpret them.
Commands, dependencies, paths, examples.	Architecture, interfaces, configuration, assumptions.	Metrics, plots, tests, comparisons, limitations, critical reflection.
Practical usage.	Software structure.	Empirical and critical evaluation.

Recommended Document Structure

- ➊ **Project goal and prediction task**: what is predicted, for whom, and why?
- ➋ **Data and labels**: event log, prefixes, target construction, time series/model/group attributes if relevant.
- ➌ **Implementation summary**: pipeline, modules, configuration, prototype.
- ➍ **Experimental design**: split, baselines, models, parameters, metrics.
- ➎ **Results**: tables, plots, qualitative examples, runtime/resource measurements.
- ➏ **Testing evidence**: unit tests, integration tests, manual tests, edge cases.
- ➐ **Critical evaluation**: interpretation, limitations, threats to validity, improvements.
- ➑ **Project retrospective**: what worked, what did not, what was learned.

Start From the Assignment Question

- Your assessment should be organized around the **main project question**.
- Examples:
 - Remaining time: *How accurately can we predict the remaining cycle time of an active case?*
 - Explainability: *Can we predict the outcome and explain predictions in an understandable way?*
 - Fairness: *Does the model behave differently for relevant groups, and can we reduce harmful gaps?*
 - Time series: *Does adding aligned time-series information improve prediction?*
 - Conformance predictions: *Does conformance information improve future deviation-risk prediction?*
- Do not only report what you implemented. Report what you learned from evaluating it.

Define the Prediction Unit and Target

- State clearly what one row in your supervised dataset represents:
 - one completed case,
 - one prefix of a running case,
 - one time step or window,
 - one case-prefix enriched with external information.
- State the target variable and how it is constructed:
 - regression target, e.g. remaining time in hours,
 - binary classification target, e.g. late vs. on-time,
 - risk score, e.g. final deviation cost above a threshold.
- Document any filtering rule: incomplete cases, very short prefixes, outliers, missing labels.

Data Description: Minimum Information

- Identify the **source and version** of every dataset used.
- For each event log, report at least:
 - number of cases and events,
 - number of activities,
 - time span,
 - average and distribution of case length,
 - relevant attributes used as features or groups.
- If applicable, also describe:
 - external time series: frequency, missing values, alignment strategy,
 - process model: notation, source, discovery/import method,
 - synthetic data: generation rules and why they are realistic enough.

Prefix Construction and Labels

- For prefix-level prediction, define how prefixes are generated:
 - after every event,
 - after selected prefix lengths,
 - after time windows,
 - with a maximum prefix length or sampling rule.
- Remaining-time example:

$$y_{c,k} = t_{\text{complete}}(c) - t_{\text{last}}(c, k)$$

where c is the case and k is the prefix length.

- Outcome-prediction example:
 - every prefix of a case receives the final case outcome as label,
 - but features must contain only information available up to that prefix.

Leakage-Safe Splitting

- Use a split that reflects the operational setting:
 - preferably **time-based by case start or completion time**,
 - or a group split that keeps all prefixes of one case together.
- Do **not** randomly split prefix rows if prefixes from the same case can appear in different sets.
- Separate roles of the splits:
 - training: fit model and preprocessing,
 - validation: tune hyperparameters and thresholds,
 - test: final, once-only estimate of performance.
- State the exact split dates, percentages, or case counts.

Baselines and Competing Approaches

- Always compare your main approach to a **simple baseline**.
- Good baselines depend on the task:
 - regression: training-set mean, median, or current-activity conditional mean,
 - classification: majority class, simple rule, logistic regression,
 - explainability: interpretable model vs. black-box model with SHAP/LIME,
 - fairness: model before vs. after mitigation,
 - time series: log-only model vs. log+time-series model,
 - conformance: log-only model vs. conformance-aware model.
- Report implementation details and versions when relevant.
- A model is convincing only if it improves on a meaningful reference point or explains why it does not.

- Many results depend on choices that should be visible and reproducible:
 - prefix lengths or time-window sizes,
 - SLA or deviation thresholds,
 - selected feature sets,
 - model hyperparameters,
 - random seeds,
 - fairness mitigation or explanation settings.
- Keep key parameters in one place:
 - `config.py`, `config.yaml`, or a clearly marked notebook cell.
- In the document, explain which parameters were varied and which were fixed.

Choosing a Good Evaluation Set

- The test set should cover the situations your system is supposed to handle.
- Include variation when possible:
 - short and long cases,
 - frequent and less frequent variants,
 - early and late prefixes,
 - small and large groups for fairness analysis,
 - low-load and high-load periods for time-series projects,
 - conforming and deviating traces for conformance projects.
- If the provided log is small, use careful qualitative analysis and avoid overclaiming.
- If you add synthetic logs or generated cases, document exactly how they were produced.

Data Formats and Data Contracts

- State which input formats your pipeline supports:
 - .xes: standard event-log format,
 - .csv: easy to inspect and exchange,
 - .parquet: efficient for large tabular feature data.
- Define the minimal schema expected by the code:
 - case identifier,
 - activity name,
 - timestamp,
 - target-related attributes,
 - optional group, conformance, or time-series keys.
- Include a tiny valid example in the repository for testing and demonstration.

- Use regression metrics for remaining time, duration, waiting time, or deviation-score prediction.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- **MAE**: easy to interpret in the original unit, e.g. hours.
- **RMSE**: penalizes large errors more strongly.
- Optional: median absolute error, R^2 , MAPE if the target makes it meaningful.
- Always state the unit of the target and error values.

Classification Metrics

	Predicted positive	Predicted negative
Actual positive	True positive (TP)	False negative (FN)
Actual negative	False positive (FP)	True negative (TN)

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

- Also report ROC AUC, PR AUC, accuracy, or Brier score when appropriate.
- For imbalanced tasks, precision, recall, PR AUC, and calibration are often more informative than accuracy alone.

Evaluate by Prefix Length

- In predictive monitoring, performance at different case stages matters.
- Report results for meaningful prefix groups, for example:
 - **early**: first 1–2 events,
 - **middle**: around 5 events or 25–50% of the case,
 - **late**: 10 events or 75% of the case.
- Explain how these groups were defined for your log.
- A strong model should not only perform well late, when most of the case is already known.
- Good analysis: *Where does the model become useful enough for an operational decision?*

- For SHAP/LIME projects, evaluate both **predictive quality** and **explanation quality**.
- Include:
 - model performance on the test set,
 - global explanation: top features and their direction of influence,
 - local explanations for selected cases,
 - comparison to an interpretable model such as logistic regression or a small decision tree.
- Pick local examples deliberately:
 - one correct positive,
 - one correct negative,
 - one false positive or false negative.
- Discuss whether explanations match process intuition or reveal suspicious shortcuts.

- Fairness projects need overall metrics and **group-wise** metrics.
- For at least two groups, report:
 - accuracy,
 - positive prediction rate,
 - true positive rate,
 - false positive rate.
- Compute gaps, for example:
 - demographic-parity gap: difference in positive prediction rates,
 - equal-opportunity gap: difference in true positive rates,
 - false-positive-rate gap.
- Compare before and after mitigation and discuss the accuracy–fairness trade-off.

Time-Series-Enhanced Assessment

- The key comparison is:
 - **log-only baseline** vs. **log + aligned time-series features**.
- Document the time-series preprocessing:
 - resampling frequency,
 - missing-value handling,
 - lagged values or sliding-window statistics,
 - synchronization with prefix timestamps.
- Analyze under which conditions the additional signal helps:
 - high workload periods,
 - specific activities,
 - early vs. late prefixes,
 - different prediction horizons.
- Also report the computational and implementation overhead.

Conformance-Aware Assessment

- The key comparison is:
 - **log-only baseline** vs. **log + conformance diagnostics**.
- State clearly:
 - which model is used for conformance checking,
 - how labels are defined, e.g. final alignment cost above a threshold,
 - which diagnostics become features: alignment cost, missing moves, inserted moves, marking/state.
- Evaluate both predictive benefit and cost:
 - Does conformance information improve early warning?
 - Which diagnostics are most useful?
 - Is runtime feasible for online monitoring?

Representing Numbers in the Assessment Document

- Numbers are outputs of automatic experiments and tests:
 - prediction metrics,
 - fairness gaps,
 - explanation summaries,
 - runtime and memory measurements,
 - test results.
- Tables are precise, but often hard to read in isolation.
- Every important table should be accompanied by:
 - one sentence explaining the main takeaway,
 - a plot when patterns matter,
 - the configuration that produced the result.
- Do not hide bad results. Explain them.

- **Performance by prefix length**: MAE, F1, AUC, or calibration vs. prefix length.
- **Prediction error distribution**: histogram, boxplot, residual plot.
- **Predicted vs. actual**: especially useful for regression.
- **ROC / PR curves**: useful for classification under thresholds and imbalance.
- **Group-wise bar charts**: needed for fairness analysis.
- **Feature importance / SHAP summary**: useful for explainability and model diagnostics.
- **Runtime plots**: runtime vs. number of cases, events, prefixes, or features.

Choosing Scales and Units

- Use **linear scales** when values are directly comparable and not extremely skewed.
- Use **logarithmic scales** when values vary by orders of magnitude, e.g. runtime on small vs. large logs.
- Always label axes with units:
 - hours, days, seconds,
 - prefix length,
 - number of events or cases,
 - probability or percentage points.
- If you transform a target or feature, explain the transformation and how results are converted back.
- Avoid plots without captions or interpretation.

Runtime and Resource Measurements

- Assessment is not only about prediction accuracy.
- Measure time-consuming parts separately:
 - loading and preprocessing,
 - prefix generation,
 - feature extraction,
 - conformance checking or time-series alignment,
 - model training,
 - prediction for one new case or prefix.
- Report the machine/environment used for timing.
- Discuss whether the prototype could be used interactively or only as an offline batch analysis.

Testing Evidence

- Testing evidence should make the instructors confident that your core logic is correct.
- Combine:
 - **unit tests**: small functions and components,
 - **integration tests**: pipeline components together,
 - **manual tests**: prototype and important user scenarios,
 - **sanity checks**: expected behavior on small synthetic data.
- In the assessment document, report:
 - what was tested,
 - what data was used,
 - how to run the tests,
 - whether they pass,
 - known gaps in test coverage.

Unit Tests Expected in This Lab

- Good test targets include:
 - label construction on a tiny synthetic log,
 - prefix generation: number, ordering, and case identity,
 - train/validation/test split: no case overlap and correct time order,
 - feature matrix: expected columns, shapes, no unexpected missing values,
 - model training smoke test on toy data,
 - metric computations: fairness gaps, regression errors, or classification scores,
 - SHAP/LIME or conformance feature extraction smoke tests when relevant.
- A few meaningful tests are better than many tests that only check that files exist.
- The test data should be small enough to understand by inspection.

Reproducibility Contract

- Another person should be able to reproduce your main results from the repository.
- Provide:
 - `requirements.txt`, `environment.yml`, or `pyproject.toml`,
 - exact Python and relevant library versions,
 - data placement or download instructions,
 - one command or notebook path to run the full experiment,
 - fixed random seeds where meaningful,
 - saved metrics and figures generated by the scripts.
- Avoid hidden manual steps such as editing paths inside notebooks.
- If data cannot be committed, provide a clear placeholder structure and instructions.

Suggested Repository Structure

```
project/
  data/                # raw/prepared data; usually not committed
  src/
    data_loading.py
    labels.py
    features.py
    models.py
    evaluation.py
    prototype.py
    config.py          # or config.yaml
  tests/
    data/tiny_log.csv
    test_labels.py
    test_prefixes.py
    test_splits.py
    test_pipeline_smoke.py
  figures/
  artifacts/           # trained models, metrics, generated outputs
  notebooks/          # exploration and demos, not hidden core logic
  README.md
  ARCHITECTURE.md
  requirements.txt
```

Prototype Evidence

- The prototype does not need to be a polished product.
- It must show how the implemented pipeline would be used on new data.
- Acceptable forms:
 - command-line script,
 - Jupyter notebook with a clean demo section,
 - small Streamlit/FastAPI interface if already implemented.
- Demonstrate at least one realistic scenario:
 - load trained artifact,
 - create features for a new case or prefix,
 - produce prediction and, if relevant, explanation/fairness/conformance context,
 - print or visualize the result.

Deployment Sanity Check

- Try to run the project from a machine or environment different from your own laptop.
- This often exposes problems such as:
 - hard-coded absolute paths,
 - missing dependency versions,
 - data files assumed to be in a private folder,
 - web service only listening on 127.0.0.1,
 - notebooks relying on previous hidden state.
- If you use Docker, Compose, Streamlit, or FastAPI, document the exact commands.
- In the assessment document, state what deployment or execution check was performed.

- Provide evidence that you inspected the behavior of your own system.
- Useful debug information:
 - examples of generated prefixes and labels,
 - feature vectors for small cases,
 - model predictions for easy synthetic examples,
 - conformance diagnostics for conforming and non-conforming traces,
 - SHAP/LIME explanations for selected examples,
 - fairness metric calculations on a toy dataset with known outcome.
- Ask: *Do two similar cases lead to similar outputs?*
- Ask: *Do clearly different cases lead to meaningfully different outputs?*

- Do not only report tables and graphs. Interpret them.
- Explain how performance depends on:
 - prefix length,
 - selected features,
 - model type and hyperparameters,
 - dataset period or variant type,
 - groups, time-series conditions, or conformance state.
- Clearly state:
 - strong points of the approach,
 - criticalities and failure modes,
 - threats to validity,
 - improvements that would be tried next.

Threats to Validity

- **Data validity:** Is the log complete, representative, and correctly timestamped?
- **Label validity:** Does the label really represent the business concept you claim?
- **Evaluation validity:** Does the split avoid future information and case leakage?
- **External validity:** Would the result generalize to another period, organization, or log?
- **Implementation validity:** Could bugs in preprocessing, feature engineering, or metrics affect conclusions?
- **Interpretation validity:** Are explanation, fairness, or conformance results overinterpreted?

Project Retrospective

- The final phase review should reflect on the project as a software project.
- Discuss:
 - what worked according to the initial requirements and goals,
 - what did not work as planned,
 - which requirements changed and why,
 - what your team learned technically and organizationally,
 - which risks were discovered late,
 - what you would do differently in another iteration.
- This is not a place to blame; it is a place to show professional reflection.

Common Mistakes to Avoid

- No baseline, or only comparing complex models with each other.
- Random row split that leaks prefixes of the same case across train and test.
- Reporting validation results as final test results.
- Tables and plots without interpretation.
- Unclear target definition or changing thresholds without explanation.
- Evaluation notebook that works only on one laptop.
- Tests that do not check the important logic.
- Only reporting strong points and hiding limitations.

Final Checklist Before Submission

- The assessment document states the task, data, labels, split, metrics, baselines, and configurations.
- Main results are reproducible from a script or clearly documented notebook.
- Tables are accompanied by plots and interpretation.
- Tests cover the most important logic and can be run with one command.
- The prototype can load a trained artifact and process a new case or prefix.
- Limitations and threats to validity are explicitly discussed.
- README, architecture documentation, and requirements are up to date.
- The Git repository contains the final code, generated figures, and instructions.

Take-Home Message

- The assessment document should make your project **inspectable**.
- A good submission shows:
 - what was built,
 - how it was tested,
 - how it was evaluated,
 - what the results mean,
 - where the approach is still limited.
- The strongest projects are not those that claim perfection, but those that provide reliable evidence and honest interpretation.